



Von Text zu virtuellen 3D Welten

Seminarabschlussarbeit der Gruppe Bild zu 3D Szene

vorgelegt von:

Frederik Flügge

Franka Freericks

David Metzdorf

Jonas Pollok

Lehrende:

Prof. Dr.-Ing. Lars Linsen

Gabriel Borelli

Janin Uedemann

Münster, 7. März 2026

Inhaltsverzeichnis

1. Einleitung	1
1.1 Motivation (Frederik)	1
1.2 Zielsetzung der Arbeit (Frederik)	1
1.3 Organisation und Zusammenarbeit (Frederik)	1
1.4 Aufbau der Arbeit (Frederik)	2
2. Grundlagen und verwandte Arbeiten	2
2.1 Text-zu-Text Modelle	2
2.1.1 GLM4.7 (David)	2
2.2 Text-zu-Bild Modelle	3
2.2.1 Z-Image (David)	3
2.3 Text-zu-Multiview Modelle	3
2.3.1 FLUX.2 (David)	4
2.3.2 MVAdapter (Frederik)	4
2.3.3 MVDiffusion (Franka)	5
2.4 Video-Generierungsmodelle	5
2.4.1 LTX-2 (Franka)	5
2.4.2 Panorama zu Video (Franka)	6
2.5 Bild/Video-zu-3D Modelle	6
2.5.1 WorldGen (Jonas)	6
2.5.2 WonderWorld (Franka)	7
2.5.3 ZeroScene (Frederik)	7
2.5.4 MIDI-3D (Jonas)	8
2.5.5 HunyuanWorld-Mirror (David)	8
2.6 Sonstige Tools	9
2.6.1 Open3D (David)	9
2.6.2 Blender (Jonas)	9
2.6.3 Unity (Frederik)	9
2.6.4 Unreal Engine (Franka)	10
3. Evaluierte Alternativ-Pipelines	10
3.1 Bildgenerierung	10
3.1.1 FLUX.2 (David)	10

3.1.2 MVAdapter (Frederik)	11
3.2 2D-Vorlage zur 3D-Szene	14
3.2.1 WorldGen (Jonas)	14
3.2.2 MIDI-3D (Jonas)	18
3.3 Post-Processing und Umsetzung in Unity	20
3.3.1 Unity PLY Import (Frederik)	20
3.3.2 Ball Pivoting (Jonas)	20
3.3.3 Vertex-Farben (Jonas)	21
4. Konzeption und Implementierung der Pipeline	22
4.1 Die Gesamtpipeline (Jonas)	22
4.2 Schritt 1: Prompt-Verbesserung	23
4.2.1 GLM4.7 (David)	23
4.2.2 Bildgenerierung (David)	23
4.2.3 Videogenerierung (Frederik)	24
4.3 Schritt 2: Bildgenerierung mit Z-Image (David)	26
4.4 Schritt 3: Von Bild + Text zu Video (Franka)	27
4.5 Schritt 4: Von der 2D-Vorlage zur 3D-Szene (David)	27
4.6 Schritt 5: Post-Processing und Umsetzung in Unity (Jonas)	29
4.6.1 Verarbeitung der Punktwolke mit Open 3D (Jonas)	30
4.6.2 Import und Interaktion in Unity (Jonas)	33
5. Ergebnisse der finalen Pipeline	36
6. Fazit (Franka)	39

1. Einleitung

1.1 Motivation (Frederik)

Mit künstlicher Intelligenz virtuelle 3D Welten erzeugen – was vor ein paar Jahren noch nach einer utopischen Vorstellung klang, ist mittlerweile Realität geworden. Neben der alltäglichen Arbeit mit KI, wie z.B. der Generierung von Texten, lassen sich mittlerweile auch ganze Bilder, Videos oder auch 3D Welten erzeugen.

KI-generierte 3D-Welten haben großes Potential. Beispielsweise lassen sich Konzepte oder Entwürfe auf schnelle Art und Weise realisieren oder sogar ganze Prototypen Digitaler Zwillinge generieren, was unter Anderem in der Architektur oder Industrie Anwendung finden kann. Die Generierung von 3D-Welten aus einem einzelnen Textprompt heraus zeigt uns außerdem etwas über die kreative Arbeit von KI-Modellen.

1.2 Zielsetzung der Arbeit (Frederik)

Das Ziel dieser Arbeit ist die Zusammenstellung einer funktionsfähigen Pipeline, die verschiedene KI-Modelle miteinander kombiniert, um ausgehend von einem einzelnen Textprompt eine erlebbare virtuelle 3D Szene zu generieren.

Dabei untersuchen wir insbesondere, was Stärken und Schwächen jedes einzelnen Modells sind und wie verschiedene Modelle unterschiedlich gut miteinander kombiniert werden können.

1.3 Organisation und Zusammenarbeit (Frederik)

Unsere Arbeit im Team war so strukturiert, dass wir zusätzlich zur wöchentlichen Seminarsitzung ein weiteres Treffen innerhalb unserer Gruppe hatten, in dem wir unsere Ergebnisse besprochen haben. Außerdem haben wir überlegt, wie wir diese Ergebnisse am besten im Seminar präsentieren können. Am Ende jedes Seminartreffens haben wir grobe Arbeitspakete entwickelt, sodass jeder von uns eigenständig in seinem Aufgabenbereich weiterarbeiten konnte.

Darüber hinaus haben wir dem Seminar ein Miro-Board zur Verfügung gestellt, auf denen wir unsere Ansätze zur Entwicklung der Pipeline visuell darstellen konnten. Häufig war das Miro-Board der Dreh- und Angelpunkt, auf dem viele Ergebnisse festgehalten wurden.

Das Seminar wurde außerdem in Zusammenarbeit mit Studierenden aus der Germanistik durchgeführt, die uns Texte für die Generierung von 3D-Szene zur Verfügung stellten.

1.4 Aufbau der Arbeit (Frederik)

In der folgenden Arbeit werden wir zunächst alle Modelle auflisten, die wir im Rahmen des Seminars untersucht haben. Dabei erläutern wir jeweils kurz deren Funktionsweise und bewerten, inwiefern sie für unser Vorhaben infrage gekommen sind.

Daraufhin präsentieren wir alternative Ansätze für eine potenziell funktionierende Pipeline und begründen, warum wir diese letztlich doch verworfen haben.

Abschließend folgt die Vorstellung unserer finalen Pipeline, mit der es möglich ist, ausgehend von einem Text-Prompt eine vollständige 3D-Szene zu generieren.

Den Abschluss bildet die Präsentation unserer konkreten Ergebnisse anhand eines Beispiels sowie ein kurzes Fazit.

2. Grundlagen und verwandte Arbeiten

2.1 Text-zu-Text Modelle

Durch die Zusammenarbeit mit den Deutschdidaktik-Studenten:innen haben wir viele Texte zur Verfügung gestellt bekommen, die uns die Vorstellungen und Anforderungen an den zu generierenden Raum gezeigt haben, jedoch häufig nicht direkt nutzbar als Prompt für verschiedene Modelle waren. Daher haben wir uns zusätzlich mit der Verbesserung der uns vorliegenden Texte zu Prompts, die in Umfang (generell kürzer) und Sprache (klare und objektive Beschreibungen) besser für die Modelle geeignet sind, beschäftigt.

2.1.1 GLM4.7 (David)

GLM4.7 [1] ist ein LLM von Z.ai, welches am 22.12.2025 als Open-Source Modell veröffentlicht wurde. GLM4.7 führt Preserved Thinking und Turn-level Thinking ein. Preserved Thinking erlaubt dem Modell alle vorherigen Anfragen und Antworten beizubehalten und in neuen Anfragen zu beachten. Turn-level Thinking erlaubt dem Modell die eigene Argumentation einzustellen, dabei werden einfache Fragen schneller beantwortet

und bei schweren Fragen länger nachgedacht. Somit konnte das Modell auch mit Closed-Source Modellen in einigen Benchmarks mithalten.

GLM4.7 ist ein nützliches und flexibles Tool, welches man eigentlich immer nutzen kann, wenn man mit Texten arbeitet. Deswegen haben wir uns während des Projektes sehr intensiv mit diesem Modell auf Palma auseinandergesetzt.

2.2 Text-zu-Bild Modelle

2.2.1 Z-Image (David)

Z-Image-Turbo [2] ist ein text-to-image Modell, welches von der Alibaba Group am 27.11.2025 als Open-Source Modell veröffentlicht wurde. Z-Image ein kosteneffizientes Diffusion Modell, das heißt es startet mit viel Noise und kann dann sehr schnell akkurate Bilder aus Textbeschreibungen generieren. Bemerke hier, dass wir konkret die Z-Image-Turbo Variante meinen, da die anderen Varianten zum Zeitpunkt des Semesters nicht veröffentlicht wurden.

Die Effizienz des Modells basiert dabei auf den vier effizienten Grundstrategien. Wir haben eine Effiziente Datenstruktur, welche Daten für den Diffusion Prozess bereithält. Effiziente Architektur durch 6B Single Stream Transformer, welcher uns erlaubt gleichzeitig an Texttokens und Imagetokens zuarbeiten. Effizientes Training in drei Phasen, wobei zuerst grobe Strukturen generiert werden und dann komplexer aufgebaut werden. Zuletzt haben wir eine effiziente Inferenz, die in nur 8 Schritte schon hochwertige Bilder generieren kann.

Z-Image ist ein verlässlicher Bildgenerator, welcher auf Palma schnell Bilder aus Textbeschreibungen generieren kann. Dabei sind diese Bilder exzellente Referenzbilder für eventuelle Bilderweiterungstools. Da dies sehr relevant für die 3D Szenen Generierung ist, haben wir uns stark mit diesem Modell beschäftigt.

2.3 Text-zu-Multiview Modelle

Bei einem Text-zu-Multiview Modell gibt man einen Text-Prompt hinein und bekommt mehrere Input Bilder heraus. Im Folgenden listen wir alle Modelle auf, die wir uns im Rahmen des Projektseminars länger angeschaut haben.

2.3.1 FLUX.2 (David)

FLUX.2 [3] ist ein text-to-image Modell, welches von Black Forest Labs am 25.11.2025 als Open-Source Modell veröffentlicht wurde. Dabei akzeptiert das Modell auch bis zu 10 Referenzbilder als optionalen Eingang für die Bildgenerierung. Diese Funktion können wir dann auch zum Beispiel für Bildänderungen nutzen, also Inpainting.

Das Modell basiert dabei auf der latent flow matching Architektur statt der klassischen Diffusion, welches uns dann erlaubt Bildgenerierung und Bildänderung zu einer Architektur zu kombinieren. Konkret wird Mistral-3-24B Vision-Language-Modell (VLM) mit einem Rectified Flow Transformer gekoppelt. Dabei liefert das VLM ein reales Weltwissen und Kontextverständnis und der Transformer liefert die Raumbeziehungen, Materialeigenschaften und Zusammensetzungslogik.

2.3.2 MVAdapter (Frederik)

Ein weiteres Modell, das wir uns im Kontext der Text-to-Multiview-(T2M)-Generierung angeschaut haben, ist "MVAdapter" (Dez 2024) [4]. Dabei handelt es sich um einen "adapter-basierten Plug-and-Play-Ansatz" für Multi-View Image Generation. MAdapter kann auf ein beliebiges Text-to-Image-(T2I)-Diffusionsmodell gesetzt werden und erweitert dieses um die Generierung von mehreren konsistenten Perspektiven eines Objekts ohne dabei die ursprüngliche Architektur des darunterliegenden Modells zu verändern.

Herkömmliche (T2M)-Methoden müssen im Vergleich häufig starke Modifikationen an dem zugrunde liegenden T2I-Modell vornehmen, was zu einer längeren Rechenzeit und schlechterer Bildqualität führt.

Das Modell eignet sich insbesondere für die Generierung mehrerer konsistenter Perspektiven eines Objekts innerhalb einer Szene, wie z.B. einem Tisch oder einer Spielzeugfigur. Die Anwendung ist dabei sehr simpel: Man gibt einen Text-Prompt, ggf. mit einem Input-Bild zusammen, als Eingabe hinein, woraufhin das Modell automatisch mehrere Ansichten desselben Motivs aus unterschiedlichen Blickwinkeln generiert.

Wir haben im Zuge unserer Arbeit diesen Ansatz in Erwägung gezogen und werden später noch einmal genauer auf unsere Verwendung des Modells eingehen.

2.3.3 MVDiffusion (Franka)

MVDiffusion [5] ist ein älteres Modell (Tang et al., 2023), welches aber interessant auf uns wirkte, da es von einem Text sowohl Multiview-Images als auch Panoramas generieren kann. Diese Funktionalität hätte uns bei der Generierung des Inputs für HunyuanWorld-Mirror (2.5.5) den Schritt eine Kamerafahrt oder erst ein Bild und basierend darauf ein Video zu generieren erspart hätte.

MVDiffusion erzeugt seine Multiview Bilder durch ein Stable Diffusion Modell, welches um einen Mechanismus erweitert wurde, der nach jedem parallelen Diffusionsschritt Pixel-zu-Pixel Korrespondenz zwischen den Multiview Bildern erkennt und die Generierung dahingegen anpasst.

Wir haben uns nach einigen Tests auf PALMA gegen dieses Modell entschieden, da es auf Gewichte trainiert wurde, die nicht mehr in ihrer Vollständigkeit verfügbar sind und die Ergebnisse, die dieses Modell erzeugt hat, daher nicht nutzbar waren.

2.4 Video-Generierungsmodelle

Nach umfangreichen Tests mit HunyuanWorld-Mirror (2.5.5) haben wir fest-gestellt, dass sich Videos besonders gut als Input für die Generierung kohärenter und vollständiger 3D-Modelle eignen.

2.4.1 LTX-2 (Franka)

LTX-2 (HaCohen et al., 2026) [6] ist erst gegen Ende unseres Projektseminars als Open-Source-Projekt erschienen und wir haben uns dazu entschieden es uns näher anzuschauen, obwohl es eher auf die Generierung cineastischer Szenen ausgelegt ist und wir für den Input für HunyuanWorld-Mirror einen eher “objektiveren” Filmstil benötigten. Insbesondere hat es unser Interesse geweckt, da es innerhalb unserer Gruppe die bis dahin besten Videos produziert hat und sowohl Text als auch Bilder als Input verarbeiten kann.

Das Modell ist ein latentes Diffusionsmodell, welches Videos mit Textprompts generiert. Diese werden zu Embedding Vektoren umgewandelt, an welchen sich das Modell im Latenten Raum orientiert. Mittels Cross-Attention Ebenen wird sichergestellt, dass das Video in sich kohärent bleibt, auch wenn einzelne Frames gleichzeitig generiert werden.

Wir haben uns dafür entschieden LTX-2 tiefer zu evaluieren, da es insbesondere auf dem Playground [7] sehr beeindruckende Ergebnisse produziert hat, jedoch auch auf PALMA vielversprechende Videos generieren konnte, die für unsere Zwecke angemessen waren.

2.4.2 Panorama zu Video (Franka)

Ein weiterer Ansatz zum Generieren von Input für HunyuanWorld-Mirror (siehe 2.5.5), den wir verfolgt haben, war es ein Panorama zu generieren, da HWM so nicht nur eine gute Basis an Informationen zu den Elementen im Raum hätte, sondern auch zu deren räumlichen Beziehungen zueinander. Da bei dem Input eines Panorama Bildes, HWM die Räumlichkeit der Szene allerdings nicht erkennt und ein verzerrtes Ergebnis liefert war die Idee einen Zwischenschritt in die Pipeline einzubauen und das generierte Panorama mittels eines Skripts (`panning_video.py`) zu einem Video umzubauen, das sich besser als Input eignen würde.

Dieser Ansatz hat sehr gut bei manuell aufgenommenen Panorama-”Fotos” funktioniert. Da leider kein Open-Source-Modell auf PALMA adäquate 360° Panoramas generieren konnte (z.B. MVDiffusion 2.3.3), haben wir die Pipeline: Bild -> Panorama -> Video -> 3D-Szene nicht umgesetzt.

2.5 Bild/Video-zu-3D Modelle

Die Erstellung von dreidimensionalen Szenen aus zweidimensionalen Bildern oder Videos war die primäre Aufgabe unserer Gruppe und stellte daher vor allem zu Beginn des Projektseminars unseren größten Arbeitsschwerpunkt dar. Unsere Auswahl der Modelle, die aus Eingabebildern oder Videos dreidimensionale Repräsentationen generieren, umfasst die von uns als vielversprechend eingeschätzten Modelle, in deren Evaluierung wir entsprechend mehr Zeit investiert haben.

2.5.1 WorldGen (Jonas)

WorldGen (Ziyang Xie, 2025) [8] ist ein Open-Source-Framework zur Generierung dreidimensionaler Szenen. Das Tool ermöglicht sowohl die Erzeugung von 3D-Szenen aus Textbeschreibungen als auch aus einzelnen Eingabebildern. Dabei sollen sowohl Innen- als auch Außenszenen in unterschiedlichen visuellen Stilen unterstützt werden.

Wir haben uns mit der Szenengenerierung aus Eingabebildern beschäftigt. Dabei ist der Grundlegende Ansatz des Tools zunächst mit FLUX.1 [9] ein 360°-Panoramabild zu erzeugen. Anschließend wird mit dem Depth-Estimation-Modell UniK3D [10] eine Tiefenkarte erstellt, welche zusammen mit dem Panorama in eine dreidimensionale Punktwolke überführt wird.

Aufgrund der sehr vielversprechenden Beispiele im GitHub und soliden ersten Ergebnissen haben wir uns intensiver mit dem Framework beschäftigt und dieses für die Pipeline in Betracht gezogen.

2.5.2 WonderWorld (Franka)

WonderWorld (Hong-Xing et al., 2025) [11] ist ein Modell, das in seiner Funktionalität sehr nah an unsere ursprüngliche Zielsetzung des Projektseminars kommt. Das Modell nimmt ein Bild als Input und generiert darauf dynamisch den Rest der Szene, basierend darauf, wo der Nutzer die “Kamera” neigt. Es ist also möglich die Szene interaktiv zu generieren und die nächste Ansicht darüber hinaus mittels Texteingaben weiter zu spezifizieren.

WonderWorld schafft diese Interaktivität, indem mittels Guided Depth Diffusion basierend auf dem Input Bild eine erste 3D-Szene generiert wird. Bei Kamera Bewegung dient ein Ausschnitt von dieser, sowie der Nutzer-Text Input, als Input für die Generierung eines weiteren Bildes. Mit diesem kann dann eine weitere Depth Map als Erweiterung der bisherigen erstellt werden.

Wegen massiver Probleme auf PALMA (insbesondere mit dem PyTorch3D-Import) haben wir uns gegen dieses Modell entschieden.

2.5.3 ZeroScene (Frederik)

ZeroScene [12] ist ein Zero-Shot Framework zur Generierung vollständiger 3D-Szenen auf Basis eines einzelnen Eingabebildes. “Zero-Shot” bedeutet in diesem Kontext, dass das Modell eine neue Szene generiert, ohne davor spezifische Trainingsdaten und Beispiele gesehen zu haben. Darüber hinaus ermöglicht dieses Modell im Nachhinein Texturen per Textprompt zu bearbeiten.

ZeroScene separiert hierfür zunächst die Objekte des Eingabe-Bildes vom Hintergrund und baut diese anschließend als 3D-Modelle nach. Die zuvor in 2D extrahierten Objekte werden danach mit den generierten 3D-Objekten kombiniert. Da beide Repräsentationen ihre genaue

Position im ursprünglichen Bild speichern, können daraufhin die Objekte wieder an derselben Stelle in den Hintergrund eingefügt werden. Dadurch wird eine konsistente Darstellung innerhalb einer Szene ermöglicht. Im Gegensatz zu vielen bestehenden Methoden entstehen auf diese Weise konsistente Szenen ohne geometrische Überlappungen oder fehlerhafte Objektplatzierungen.

Da dieses Modell aber leider nicht OpenSource zur Verfügung steht, konnten wir es nicht für unser Vorhaben in Erwägung ziehen.

2.5.4 MIDI-3D (Jonas)

MIDI (Zehuan Huang et al., 2025) [13] [14] ist ein Multi-Instance Diffusion Modell zur Erzeugung dreidimensionaler Szenen aus einem einzelnen Eingabebild. Dabei unterstützt das Modell sowohl realistische als auch stilisierte Eingabebilder und erzeugt eine 3D-Szene, die alle dargestellten Objekte beinhaltet.

Der Ansatz von MIDI unterscheidet sich grundlegend von den Methoden anderer Modelle/Frameworks, wie WorldGen (2.5.1) oder HunyuanWorld-Mirror (2.5.5). Anstatt die gesamte Szene als Ganzes in 3D zu überführen, wird das Eingabebild zunächst mit dem Segmentierungsmodell Grounded SAM [15] in einzelne Objekte unterteilt. Anschließend werden mit dem Multi-Instance Diffusion aus allen erkannten Objekten parallel dreidimensionale Repräsentationen erzeugt, dabei wird zudem auch das MVAdapter Modell (2.3.2) [4] verwendet. Über einen speziellen Aufmerksamkeitsmechanismus (Multi-Instance Attention) werden die räumlichen Beziehungen der Objekte zueinander berücksichtigt und diese so korrekt zueinander positioniert. Das Modell generiert dabei verdeckte Bereiche der Objekte selbstständig und vervollständigt diese.

Aufgrund des anderen Ansatzes und den daraus eventuell resultierenden Vorteilen, sowie guten ersten Ergebnissen haben wir uns intensiver mit dem Modell beschäftigt.

2.5.5 HunyuanWorld-Mirror (David)

HunyuanWorld-Mirror [16] ist ein video-to-3D und multi-view-to-3D Modell, welches am 23.11.2025 von Tencent als Open-Source Modell veröffentlicht wurde. Im Wesentlichen handelt es sich um ein reines multi-view-3D Modell, dabei wird das Input Video in einzelne Frames zerteilt, welche als Input Images dienen.

Das Modell nutzt das Prinzip der Attention, um globale Zusammenhänge zwischen den verschiedenen Kameraperspektiven zu erkennen. Dabei werden durch den Multi-Modal Prior Prompting Mechanismus weitere Informationen, die man als Input mitgeben kann, mit den Images verknüpft. Das letzte Hauptbestandteil des Modells ist die Universal Geometric Prediction Architektur, es erlaubt das Modell, durch den davor gewonnen Informationen, eine passende 3D Struktur zu generieren.

HunyuanWorld-Mirror ist ein Modell, welches in der Lage zu verlässliche 3D Szenen zu generieren. Somit ist zählt es zu den Modellen, mit denen wir uns am meisten beschäftigt haben.

2.6 Sonstige Tools

2.6.1 Open3D (David)

Open3D [17] ist eine Library, welche am 22.02.2018 von Intel Labs als Open-Source Tool veröffentlicht wurde. Innerhalb der Library finden wir sämtliche Funktionen und vortrainierte Modelle für die Bearbeitung von 3D Szenen. Zum Beispiel finden wir hier mehrere Algorithmen für die Umwandlung einer Punktwolke zum Mesh.

2.6.2 Blender (Jonas)

Blender [18] ist eine Open-Source-Software für 3D-Modellierung, Animation, visuelle Effekte und Rendering. Die Software wurde bereits 1998 von NaN veröffentlicht und seit 2002 von der Blender Foundation weiterentwickelt. Blender ist eines der am weitesten verbreiteten Werkzeuge im Bereich der 3D-Gestaltung.

Neben den umfangreichen Funktionen bietet Blender auch nativ einige Methoden zur Konvertierung verschiedener Dateitypen, wie .ply, .obj oder .fbx, was die Software für das Post-Processing in unserer Pipeline interessant gemacht hat.

2.6.3 Unity (Frederik)

Unity [19] ist eine plattformübergreifende Echtzeit-Entwicklungsplattform zum Erstellen interaktiver 2D-, 3D- sowie VR/AR-Anwendungen. Die Engine wird häufig in der Spieleentwicklung, aber auch in der Simulation, Visualisierung und Forschung eingesetzt.

Im Rahmen unseres Projektseminars haben wir Unity verwendet, um die mit den anderen Modellen generierten 3D-Ergebnisse zu visualisieren und uns darin zu bewegen. Damit bietet Unity eine technische Grundlage für die Darstellung unserer Resultate in einer virtuellen 3D-Welt.

2.6.4 Unreal Engine (Franka)

Unreal Engine [20] ist analog zu Unity (2.6.3) eine Videospiel-Engine, die es ermöglicht 3D-Welten als Punktwolken oder Meshes zu importieren und diese mittels VR-Brille oder Controller/ Pfeiltasten zu betrachten. Obwohl wir Unity schon sehr gut für unsere Zwecke funktioniert hat, wollten wir testen, ob Unreal Engine aufgrund seiner stärkeren Render-Engine noch besser geeignet wäre. Wegen dieser Engine haben wir uns allerdings im Endeffekt gegen Unreal Engine entschieden, da es zu ressourcenhungrig war und daher zu langsam auf unseren Laptops gelaufen ist, um wirklich in Erwägung gezogen zu werden.

3. Evaluierte Alternativ-Pipelines

3.1 Bildgenerierung

3.1.1 FLUX.2 (David)

Ein weiteres Tool, welches wir uns genau angeschaut haben, ist FLUX.2 [3]. Mit FLUX.2 kann man Bilder aus Textbeschreibungen generieren, somit steht es direkt in Konkurrenz mit Z-Image (2.2.1) für die endgültige Pipeline. Beide Modelle liefern hervorragende Bilder, welche zu den Textbeschreibungen passen. Somit ist dieses Modell tatsächlich austauschbar mit Z-Image und wir bekommen ähnlich gute Ergebnisse. Zudem ist FLUX.2 in der Lage mehrere Perspektiven mit dem Inpainting zu generieren, woraus man direkt 3D Szenen mit HunyuanWorld-Mirror (2.5.5) generieren kann.

Letztendlich haben wir uns jedoch für Z-Image entschieden als Bildgenerator für die Pipeline. Des einem ist Z-Image einfacher gewesen auf Palma zum Laufen zu bekommen, somit konnten wir früher mit dem Testen von Z-Image starten. Zudem nutzen wir die Turbo Variante von Z-Image, es ist eine Standardvariant angekündigt und die Hoffnung war, dass diese noch während des Semesters veröffentlicht wird und bessere Ergebnisse als die Turbo Variante liefert. Um die Variante von Z-Image zu tauschen, braucht man auch nur auf dem Ort der gedownloadete

Variante zeigen. Aus diesen Gründen wurde Z-Image viel stärker getestet, wobei wir immer positive Ergebnisse bekommen hatten.

Des Weiteren haben wir uns auch die Bilderweiterung mit FLUX.2 angeguckt, jedoch haben wir uns dort auch für alternativen entschieden. Bei der Bilderweiterung mit FLUX.2 muss jede Perspektive separat generiert wird, somit ist der Aufwand für die Person die endgültig die 3D Szenen generieren muss viel mehr. Zudem können wir auch nach Testen mit dem Modell sagen, dass oft die Übergänge zwischen den Multiviews inkonsistent waren, oder das Modell hat sich nur auf einem Inputbild fokussiert, obwohl wir mehrere mitgegeben hatten.

Im Allgemeinen ist FLUX.2 ein tolles Modell, jedoch hatten wir schon eine gültige Pipeline zum Zeitpunkt, als wir dieses Modell auf Palma laufen konnten. Dabei waren die Ergebnisse nicht besser mit FLUX.2, sodass wir uns gegen dieses Modell entschieden haben.

3.1.2 MVAdapter (Frederik)

Für die Bildgenerierung stand uns außerdem das “MVAdapter” [4] Modell zur Verfügung. In einem ersten Versuch haben wir ein einzelnes Objekt aus dem Text der Sprachdidaktiker:innen herausgenommen und dieses ohne zusätzliches Input-Bild in das Modell gegeben. Für diese Vorgehensweise stand uns allerdings nur der „Anime-Modus“ aus der Demo-Version auf Hugging Face [21] zur Verfügung. Das erste Ergebnis war das folgende (Abb. 1 – 4):

*"A floating semi-transparent
block with text fragments"*

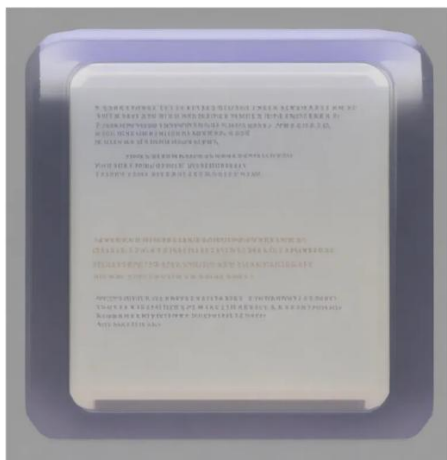


Abbildung 1

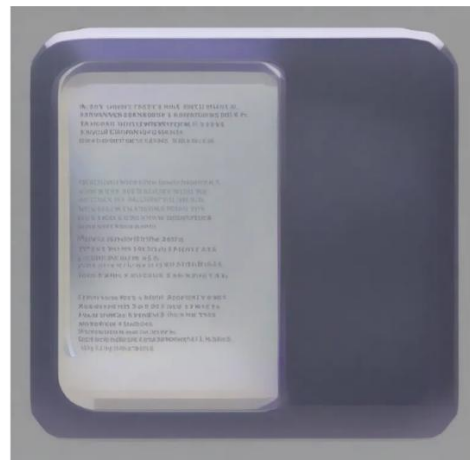


Abbildung 2

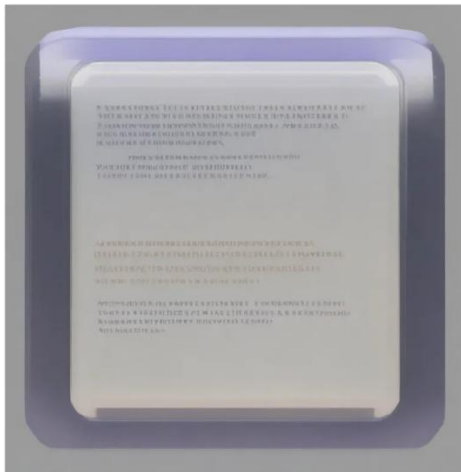


Abbildung 3

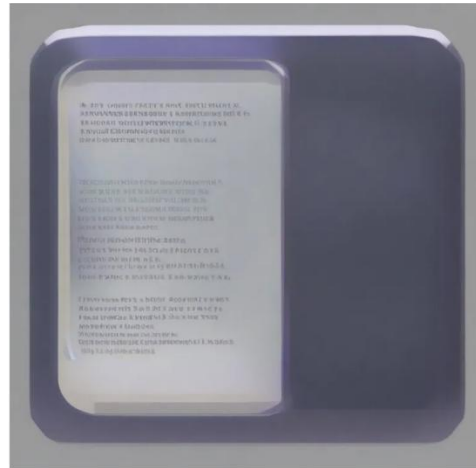


Abbildung 4

In einem zweiten Versuch haben wir dann die MultiView-Generierung anhand eines Input-Bildes (Abb. 5) durchgeführt. Hierfür haben wir ein Bild von einer Wand in einem Raum verwendet, das vorab mit Z-Image aus dem Text der Sprachdidaktiker:innen generiert worden war. Dieses Input-Bild haben wir dann mit einem durch GLM 4.7 (2.1.1) optimierten Prompt kombiniert. Zusätzlich zu diesem Prompt haben wir die beschreibenden Texte für die weiteren Wände des Raumes hinzugefügt (diese werden aus Gründen der Übersichtlichkeit im Folgenden nicht aufgeführt). Das entsprechende Ergebnis war das folgende (Abb. 6 – 11):

„You are helping to create concise, high-quality images for a 3d model generator. Split your output images into images based on each individual prompt. Make sure to keep enough overlap, such that the images are consistent in space. I gave you the Image of wall 1, generate wall 2,3,4,5 from prompt 2, 3,4,5“

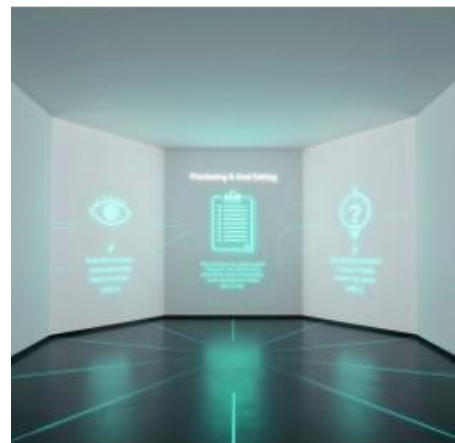


Abbildung 5



Abbildung 6



Abbildung 7



Abbildung 8



Abbildung 9



Abbildung 10



Abbildung 11

Zusammenfassend haben die Ergebnisse unserer Versuche aber leider nicht unseren Anforderungen entsprochen. Wie bereits erwähnt, ist MVAdapter primär darauf ausgelegt, mehrere konsistente Perspektiven eines einzelnen Objekts innerhalb einer Szene zu generieren. Das erklärt auch, weshalb das Ergebnis des ersten Versuchs deutlich konsistenter war. Das Modell ist leider nicht dafür konzipiert, vollständige und komplexe 3D-Szenen zu erzeugen, was im zweiten Versuch deutlich zu sehen ist.

Da wir zu diesem Zeitpunkt im Seminar ohnehin bereits Schwierigkeiten hatten, konsistente 3D-Szene zu generieren, erschien uns der Ansatz einzelne mit MVAdapter erzeugte Objekte nachträglich in eine bestehende Szene zu integrieren, als technisch zu aufwendigen Zwischenschritt.

Aus diesem Grund haben auch nicht versucht, das Modell auf PALMA zum Laufen zu bringen, und es im weiteren Projektverlauf nicht weiterverfolgt.

3.2 2D-Vorlage zur 3D-Szene

3.2.1 WorldGen (Jonas)

Als eine der ersten Methoden zur Überführung von 2D-Bildern in 3D-Szenen haben wir WorldGen [8] evaluiert. Um das Tool auf Palma zum Laufen zu bekommen, war zunächst die Entwicklung eines eigenen Setup-Skriptes notwendig, da die vorgesehenen Abhängigkeiten auf der vorhandenen Infrastruktur nicht einfach installiert werden konnten und es zu verschiedenen Kompatibilitätsproblemen kam.

Das erste Ziel war die Generierung einer Outdoorszene, dazu haben wir Eingabebilder zunächst mit Gemini 2.5 Flash [22] generiert (Abb. 12). Die generierten Szenen sahen aus einer Perspektive sehr vielversprechend aus (Abb. 13) und es war auch ein 360°-Rundumblick möglich. Dabei war der Rundumblick zwar größtenteils konsistent, aber die neu generierten Teile passten logisch nicht immer zu der eigentlichen Eingabe (Abb. 14). Sobald man die Kameraposition jedoch verschob, traten deutliche Artefakte auf (Abb. 15). Besonders problematisch bei der Generierung von Outdoorszenen war zudem die Tiefenschätzung, sodass die Punktwolke aus einer explosionsartigen Verteilung der Punkte um die Kamera entstand (Abb. 16).



Abbildung 12



Abbildung 13



Abbildung 14



Abbildung 15

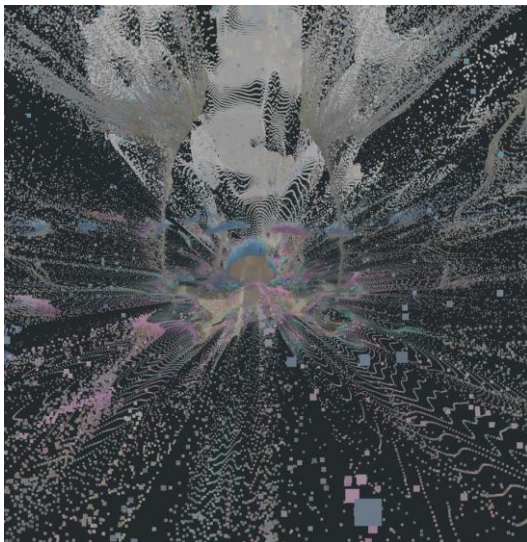


Abbildung 16

Da die Ergebnisse der Generierung von Outdoorszenen qualitativ nicht überzeugt hatten, haben wir versucht die Generierung eines einzelnen Raumes zu forcieren. Dazu haben wir Eingabebilder von der Text-zu-Bild/Video Gruppe verwendet, die klar einen Innenraum zeigen (Abb. 17). Die Ergebnisse fielen deutlich besser aus: Von einer Perspektive war das ursprüngliche Bild klar in der 3D-Szene wiederzuerkennen (Abb. 18), und es wurde ein klar abgeschlossener und farblich größtenteils passender Raum erzeugt (Abb. 19). Wie auch bei der Outdoorszene hat der neu generierte Teil nicht immer optimal zu der eigentlichen Eingabe gepasst. Zudem gab es das gleiche Problem mit der Verschiebung der Kameraperspektive (Abb. 20).



Abbildung 17



Abbildung 18

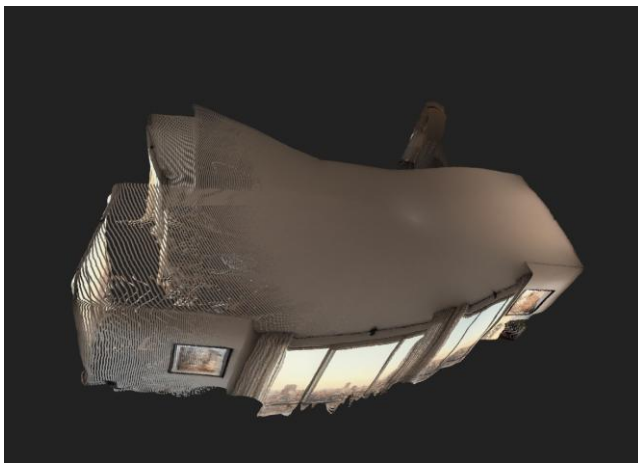


Abbildung 19



Abbildung 20

Ein letzter Ansatz, um die Artefakte zu minimieren, war die direkte Generierung eines Meshs, wodurch die Lücken, die die Artefakte verursachten, von dem Tool selbst geschlossen und ein zusammenhängendes wasserdichtes Netz erzeugt werden sollte. Das erzeugte Mesh war zwar lückenlos (Abb. 22), jedoch wurden die fehlenden Teile der Punktwolke einfach per Interpolation geschlossen, was bei den größeren Lücken zu sehr unsauberen Ergebnissen geführt hat (Abb. 23).



Abbildung 21- Eingabe Mesh Generierung



Abbildung 22



Abbildung 23

Die Generierung einer Szene war mit einer Laufzeit von etwa eineinhalb Minuten schnell, und die Ergebnisse waren grundsätzlich in die nachgelagerte Pipeline integrierbar. Jedoch haben wir uns aufgrund der eingeschränkten Kameraperspektive, den daraus resultierenden visuellen Fehlern, und der teilweise unpassenden Erweiterung der Szene dagegen entschieden WorldGen in die finale Pipeline aufzunehmen.

3.2.2 MIDI-3D (Jonas)

Als weitere lauffähige Methode zur Generierung dreidimensionaler Szenen haben wir MIDI-3D [13] evaluiert. Als Eingabe haben wir sowohl selbst mit Gemini 2.5 Flash [22] generierte Bilder (Abb. 12 – siehe 3.2.1, Abb. 24) als auch Bilder der Text-zu-Bild-Gruppe verwendet.



Abbildung 24

Vor der eigentlichen 3D-Generierung erfordert MIDI-3D eine Segmentierung des Eingabebildes (Abb. 25, Abb. 26) in einzelne Objekte. Bei dem Segmentierungsskript aus dem MIDI-3D GitHub Repository [14] konnte man Labels hinzufügen, wir konnten aber keinen Unterschied bei der Segmentierung feststellen.

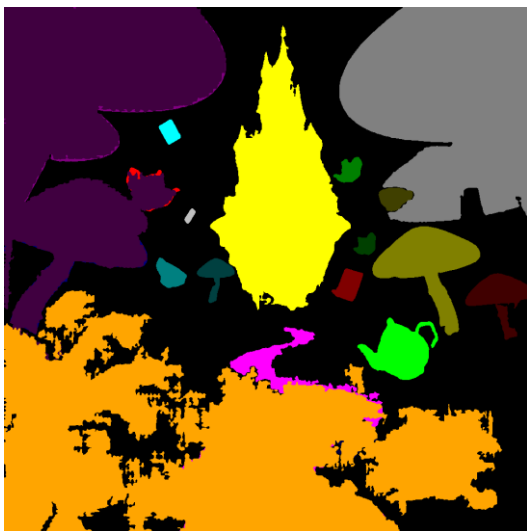


Abbildung 25

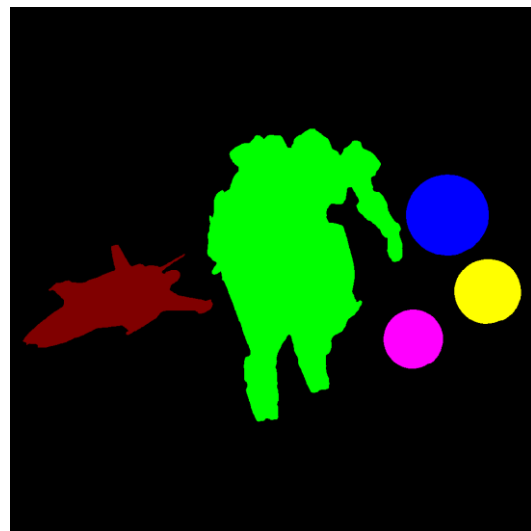


Abbildung 26

Aus der so erstellten Segmentierung und dem Ursprungsbild wurde dann jedes erkannte Objekt als 3D-Objekt erstellt und in der Szene platziert. Die Platzierung der Objekte war sowohl relativ zueinander so wie im Eingabebild zu erkennen war. Da es jedoch keine Tiefenschätzung vor dem Generieren gab, wurden Objekte, die im Hintergrund zu sehen waren, auch klein und dementsprechend nah an der Szene generiert (Abb. 27).

Die Qualität der generierten 3D-Objekte variierte stark von Objekt zu Objekt. Leider kam es bei den meisten erstellten Szenen zu einem nach hinten gedoppeltem Objekt (Abb. 28), anderen Verformungen oder Artefakten, dies liegt wahrscheinlich an den fehlenden anderen Perspektiven auf die Szene. Aufgrund dieser eingeschränkten Qualität der 3D-Objekte und der Entscheidung nicht auf MIDI-3D zu setzen, haben wir verzichtet die Funktion zur texturierten Generierung einsatzfähig zu machen.



Abbildung 27



Abbildung 28

Der Hauptgrund gegen den Einsatz von MIDI-3D in der finalen Pipeline lag darin, dass das Tool ausschließlich einzelne Objekte generiert, jedoch keinen umgebenden Raum oder Hintergrund erzeugt. Für die Erstellung einer begehbaren 3D-Szene hätte der Raum zusätzlich separat generiert und die Objekte darin platziert werden müssen. In Kombination mit der in der Regel unzureichenden Qualität der generierten Objekte aus verschiedenen Blickwinkeln erwies sich dieser Ansatz als nicht praktikabel für das Projektziel.

3.3 Post-Processing und Umsetzung in Unity

3.3.1 Unity PLY Import (Frederik)

Da Unity [19] standardmäßig keinen Import von PLY-Dateien unterstützt, mussten wir nach einer alternativen, geeigneten Lösung suchen. Dabei sind wir auf den “Unity PLY Importer” [23] gestoßen, was ein Open-Source-Package ist, das den Import von PLY-Daten in Unity ermöglicht.

Nach Installation des Packages von GitHub, war der Import von PLY-Daten möglich. Allerdings stellte sich heraus, dass die Rechenleistung beim Laden der Dateien stark beeinträchtigt wurde, wodurch Unity nahezu unbenutzbar wurde.

Vermutlich ist das Package nicht für größere Datensätze wie unsere generierten 3D-Punktwolken optimiert und stößt folglich auf Performance-Grenzen.

Außerdem war es nicht möglich, auf die importierten PLY-Dateien einen “Mesh-Collider” anzuwenden, der verhindert, dass man sich durch Objekt hindurchbewegen kann.

Insgesamt erwies sich dieser Ansatz daher als eher suboptimal.

3.3.2 Ball Pivoting (Jonas)

Wie beschrieben hatten wir Probleme damit die von den Modellen generierten Punktwolken direkt weiter zu verwenden. Daher wollten wir eine Möglichkeit finden die unabhängigen Punkte zu einem zusammenhängenden Netz zu verbinden. Für das Post-Processing haben wir primär Open3D [17] verwendet.

Für die Rekonstruktion eines Meshs aus den generierten Punktwolken haben wir parallel zwei Verfahren, den Ball-Pivoting-Algorithmus und die Poisson Surface Reconstruction (4.6.1) [24], evaluiert. Ziel war es die beiden Methoden direkt vergleichen zu können und das Post-Processing auf die Ergebnisse von HunyuanWorld-Mirror (2.5.5) zu optimieren, da zu diesem Zeitpunkt bereits absehbar war, dass dieses Modell in der finalen Pipeline zum Einsatz kommen würde.

Beim Ball-Pivoting-Algorithmus wird eine virtuelle Kugel über eine Punktwolke „gerollt“. Sobald die Kugel drei Punkte gleichzeitig berührt, wird aus diesen ein Dreieck gebildet. So

wird das Mesh lokal schrittweise aufgebaut. Die resultierenden Ergebnisse sind jedoch stark von der Gleichmäßigkeit der Punktwolke abhängig.

Die von HunyuanWorld-Mirror erzeugten Punktwolken hatten aber eine stark variierende Punktdichte. Dies führte dazu, dass der Ball-Pivoting-Algorithmus an vielen Stellen keine geschlossenen Flächen bilden konnte. Die resultierenden Netze wiesen zahlreiche und teilweise große Lücken auf, sodass die Räume nicht vollständig geschlossen waren und die 3D-Szene deutliche Lücken aufwies (Abb. 29, Abb. 30).

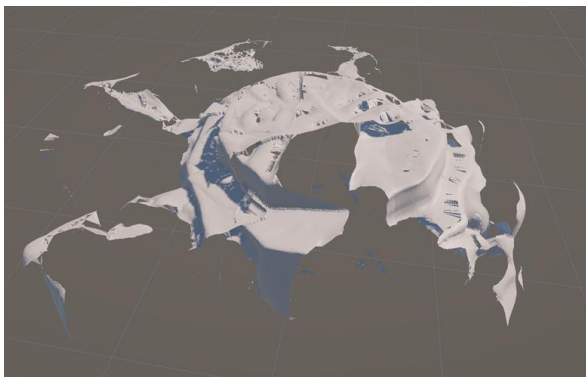


Abbildung 29



Abbildung 30

Im direkten Vergleich mit der Poisson Surface Reconstruction waren die Ergebnisse vom Ball-Pivoting-Algorithmus deutlich schlechter, da Poisson eine globale, geschlossene Oberfläche berechnet und die ungleichmäßige Punktverteilungen besser kompensiert. Aus diesem Grund fiel die Entscheidung Poisson in die finale Pipeline mit aufzunehmen und den Ball-Pivoting-Algorithmus nicht weiter zu integrieren.

3.3.3 Vertex-Farben (Jonas)

Durch die Mesh-Rekonstruktion mit Poisson wurden neue Vertices erstellt, welche keine Farbinformationen der ursprünglichen Punktwolke enthalten. Uns ist aufgefallen, dass wir einen zusätzlichen Farbübertragungsschritt in der Nachbereitung benötigen, um nicht lediglich eine farblose, graue Oberfläche zu generieren. Um die visuellen Daten der Punktwolke auf das rekonstruierte Netz zu übertragen, haben wir zunächst die Übertragung auf Vertex Farben getestet.

Wir haben dazu zunächst einen KD-Baum mit der eingebauten Open3D [25] Funktion erstellt. Mit Hilfe des Baums haben wir für jeden Vertex die nächstgelegenen Punkte der originalen

Punktwolke bestimmt. Die Farbwerte der nächsten Nachbarn wurden gemittelt und dann auf den jeweiligen Vertex übertragen. Die Farbauflösung war dementsprechend direkt an die Anzahl der Mesh-Vertices gebunden.

Um die Vertex-Farben in Unity sichtbar zu machen, mussten wir zunächst den verwendeten Shader anpassen und die Vertex-Farben direkt verwenden. Durch die geringe Farbauflösung waren die Farben auf dem Mesh verwaschen und feine Details der Punktwolke gingen verloren (Abb. 31).

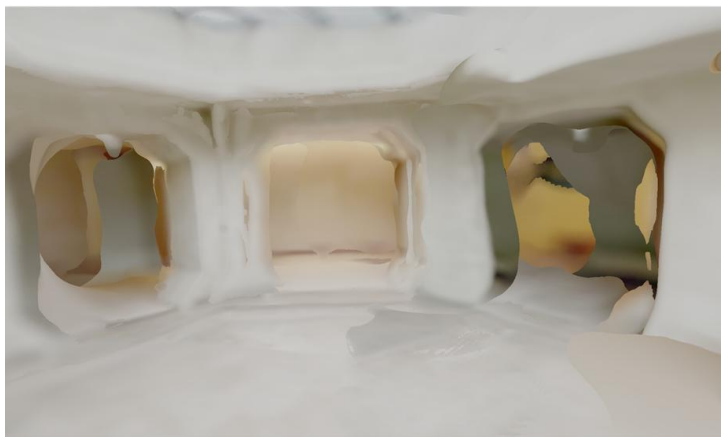


Abbildung 31

Aufgrund der eher mittelmäßigen Qualität haben wir zudem noch das Textur-Baking evaluiert (4.6.1) und die deutlich höhere Qualität hat zu der Entscheidung geführt, die Vertex-Farbübertragung nicht weiter zu verfolgen.

4. Konzeption und Implementierung der Pipeline

4.1 Die Gesamtpipeline (Jonas)

Die finale Pipeline zur Erzeugung einer begehbaren 3D-Szene aus einer textuellen Raumbeschreibung gliederte sich in fünf aufeinanderfolgende Schritte:

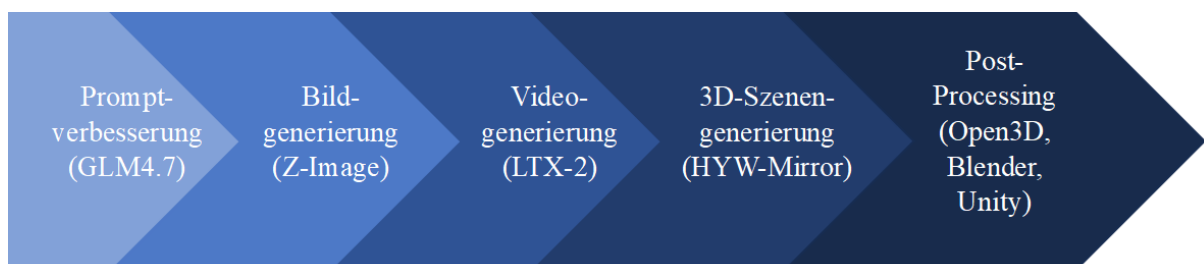


Abbildung 32

4.2 Schritt 1: Prompt-Verbesserung

4.2.1 GLM4.7 (David)

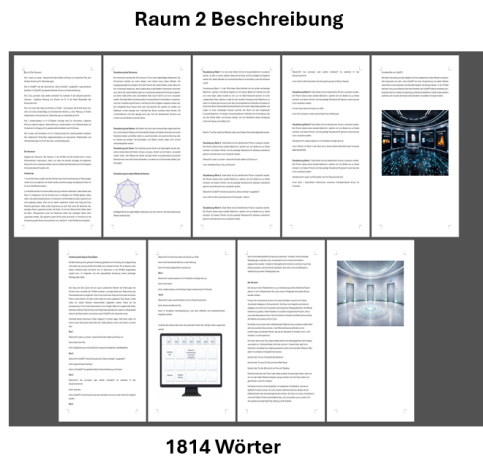
GLM4.7 [1] ist ein LLM, welches für Textbearbeitung genutzt werden kann. Mit dem Modell sind wir in der Lage, eigene Text mit Umgebungsbeschreibungen für die Pipeline zu generieren. Dies war besonders von Nutzen, als wir noch keine von den Sprachwissenschaftlern bekommen hatten.

Uns fällt auf, dass wir das Modell auch auf eine andere Weise nutzen können. Wir können das Modell als Schnittstelle für den zubekommenen Prompt und unsere endgültige Pipeline nutzen. Wir extrahieren alle für die Generierung wichtigen Informationen, um einen neuen Prompt zu formulieren, sodass wir eine klare Umgebungsbeschreibung haben. Diese Idee haben wir dann nun auch erweitert, in dem wir alle Informationen für einen spezifischen Abschnitt in der Pipeline extrahieren und damit einen neuen spezifischen Prompt schreiben, der für das Modell dieses Abschnittes abgestimmt ist. Konkreter setzen wir dies um, indem wir aus dem zubekommenen Prompt zwei abgestimmte Prompts erzeugen, einen Prompt für die Bildgenerierung und einen Prompt für die Bilderweiterung durch weitere Bilder oder Videos.

4.2.2 Bildgenerierung (David)

Die Aufteilung des, von dem Sprachwissenschaftler bekommenen, Prompt in für die Bildgenerierung Spezifizierten Prompt ist relativ einfach. Dies liegt daran, dass die von uns getesteten Bildgenerator Modelle auch gut komplexe aus Beschreibungen gute Bilder Generieren konnten, Zudem haben die Sprachwissenschaftler selbst ihre Prompts mit einem Bildgenerator getestet.

Eine weitere Besondere Bemerkung ist, dass wir mit GLM4.7 (4.2.1) gut Beschreibungen kürzen können, wobei dann mit Z-Image (4.3) bessere Ergebnisse bekommen. In Abbildung 33 ist dies beispielhaft dargestellt.



Verbesserter Prompt

An immersive, futuristic VR learning room in the shape of a regular heptagon featuring a floor, ceiling, and seven walls. Six walls act as floor-to-ceiling monitors displaying text summaries and icons; the seventh wall houses a large integrated computer screen for a task and a silver and black control panel emitting blue and white light. The space is windowless with bright lighting, dominated by white and cool blue tones. The floor is black with shimmering light blue letters and numbers resembling a system log. The dark ceiling, made of smooth carbon-like material, contains futuristic lines and patterns emitting a soft white glow. Wall 1 displays a large rectangular screen with a white document file icon against a dark background, framed by silver wall sections. Wall 2 is a dark monitor showing the headline "Lesen zu schwer – deutsche Schulen stellen auf Emojis um" above an icon of a notebook with a pen evolving into a keyboard. Wall 3 is a dark monitor showing the headline "KI macht das Leben als Schüler zur Hölle" above an icon of a computer in a burning fire. Wall 4 is a dark monitor showing the headline "Lesekompetenz ist im KI-Zeitalter wichtiger denn je" above an icon of a person holding a book facing a computer along a small timeline. Wall 5 is a dark monitor showing the headline "Lesen (und Schreiben) mit KI im Deutschunterricht" above an icon of a book with a speech bubble interacting with a computer.

198 Wörter

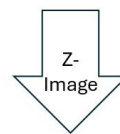
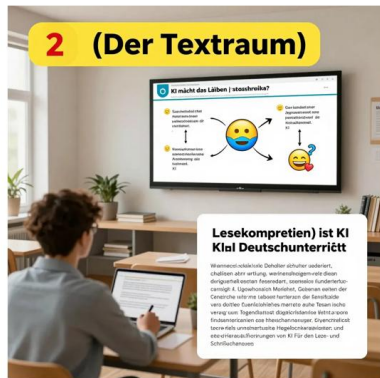
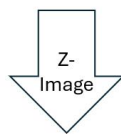


Abbildung 33

4.2.3 Videogenerierung (Frederik)

Für die Videogenerierung mit LTX-2 [6] hatten wir die Möglichkeit sowohl ein Bild als auch einen Prompt mitzugeben. Als Input Bild haben wir ein zuvor mit Z-Image generiertes Bild genutzt. Um für dieses Bild den bestmöglichen Prompt zu formulieren, haben wir uns zunächst die LTX-2 Prompting Rules [26] genauer angeschaut.

Unser größtes Problem mit anderen Videogenerierungsmodellen im Seminar war, dass es uns nicht zuverlässig gelungen ist, eine sauber 360°-Drehung innerhalb eines Raumes zu erzeugen. Eine solche vollständige Drehung der Kamera im Raum ist essenziell, um eine gute Grundlage für ein später folgendes 3D-Szenen-Generierungsmodell zu schaffen, da der Raum erst dann aus allen Perspektiven sichtbar wird.

Wenn man einen Blick auf die LTX-2 Prompting Rules wirft, fällt schnell auf, dass das Modell stark für die Generierung cinematischer Videos ausgelegt ist, was wir uns zum Vorteil machen konnten. In den Prompting Rules wird nämlich explizit beschrieben, wie Kamerafahrten formuliert werden sollten: Es ist besonders wichtig für das Modell, dass klar beschrieben wird, wann und wie sich die Kamera bewegt und auch wann sie wieder aufhört sich zu bewegen.

Mit diesen Informationen haben wir mit Hilfe von GLM 4.7 (4.2.1) den folgenden Prompt entwickelt:

"The scene begins with the camera fixed at the exact geometric center of the given room, immediately initiating a perfectly smooth, constant-speed 360-degree rotation around its vertical axis with no acceleration, no deceleration, and no lateral movement. The camera remains fully stabilized, level, and locked at eye height, rotating like a precision-controlled gimbal to reveal each wall of the room sequentially in equal intervals. The rotation completes one uninterrupted full circle with no camera shake, no motion blur distortion, and no environmental deformation."

Zudem haben wir festgestellt, dass die Qualität der Ergebnisse stark vom verwendeten Eingabebild abhängt. Bei Bildern mit vielen Objekten im Raum funktioniert der Prompt deutlich schlechter (Abb. 34). In leeren Räumen hingegen schafft die Kamera die 360°-Drehung deutlich stabiler und konsistenter (Abb. 35). Nutzt man für das Input-Bild einen Raum mit nur wenig Objekten, liefert das Modell unseren Versuchen nach in 4/5 Fällen ein zufriedenstellendes Ergebnis.



Abbildung 34



Abbildung 35

(Die Video-Ergebnisse zu den beiden Bildern finden sich in Sciebo unter *Projektseminar_Textto3D/Bild-Video-Generierung/Videos/LTX-2/Image-To-Video/Good-Bad_Result*)

4.3 Schritt 2: Bildgenerierung mit Z-Image (David)

Z-Image [2] ist ein Modell, welches Bilder aus Textbeschreibungen generiert. Dabei haben wir immer bei diesem Schritt im Hinterkopf behalten, dass wir die Bilder mit einem weiteren Tool erweitern wollen. Z-Image wurde mitten im Projektseminars veröffentlicht und galt zur veröffentlichen als ein bekanntes und hervorragendes Bildgenerierung Tool. Als wir selbst das Modell auf Palma zum Laufen bekommen haben, stellten wir fest, dass es gute und komplexe Bilder (Abb. 36, Abb. 37) aus den Beschreibungen von uns oder von GLM4.7 (4.2.1) generieren kann. Dabei sind die Bilder oft realistisch, wir sind aber in der Lage einen genauen Bild Stil anzugeben.



Abbildung 36

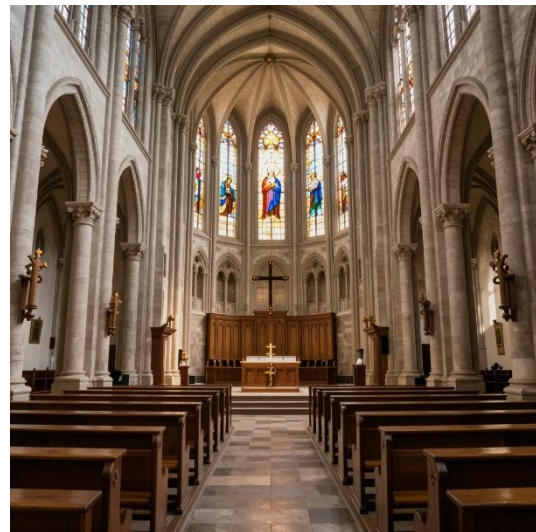


Abbildung 37

Die Bildgenerierung ist der einfachste Schritt der endgültigen Pipeline und Z-Image kann auch bei ungenauen Texten ein gutes Ergebnis liefern. Z-Image hat dabei mehrere Varianten, konkret beschreiben wir hier das Verhalten von der Turbo Variante, welche sich auf Effizienz und Schnelligkeit fokussiert, sodass man Bilder in 5 Minuten generiert bekommt. Eine Standardvariante ist angekündigt, wurde aber nicht während des Projektseminars veröffentlicht.

4.4 Schritt 3: Von Bild + Text zu Video (Franka)

Das Modell LTX-2 [6], für welches wir uns am Ende entschieden haben, besteht aus drei Hauptkomponenten, die dafür sorgen, dass Video und Audio “abhängig” voneinander kreiert werden und so eine höhere Kohärenz aufweisen. Audio und Video latente Token werden durch verschiedene VAEs generiert. In einer speziellen Embedding Pipeline werden aus dem Text-Prompt zusätzliche “Thinking Token” generiert, welches zusammen mit den Text-Token ein tieferes semantisches Verständnis fördert. Zuletzt nutzt das Modell Cross-Attention zwischen der Video- und Audio-Generierung, wodurch beide Diffusion-Pipelines aneinander angepasst werden und damit ein gut abgestimmtes Ergebnis erzielt wird. Auch wenn der Audio-Part des Modelles nicht weiter relevant für unsere Zwecke war, ist es dennoch ein integraler Teil der Architektur des Modelles.

Das Modell ist ebenfalls außerordentlich effizient und kann auf einer NVIDIA H100 GPU ein Video mit 121 Frames bei einer Auflösung von 720p jeden Diffusions-Schritt in 1,22 Sekunden durchführen. Es können außerdem Videos mit bis zu 20 Sekunden Länge generiert werden, was für uns natürlich interessant war, da ein längeres Video mehr Input-Informationen für HunyuanWorld-Mirror (4.5) enthält und so auch die 3D-Welt detaillierter wird.

Da LTX-2 erst gegen Ende des Projektseminars Open-Source erschienen ist, konnten wir nicht sonderlich viele Experimente auf PALMA starten, haben aber hauptsächlich die Pipeline `ltx_pipelines.ti2vid_two_stages`, die auch auf dem GitHub empfohlen wird genutzt und damit zwar keine perfekten, jedoch gegen Ende zufriedenstellende Ergebnisse erzielt. Videos von 5 Sekunden Länge ließen sich auf PALMA auf der NVIDIA H100 in ca. 3 Minuten vollenden. An die Ergebnisse, die der LTX-2-Playground produziert sind wir ebenfalls nicht ganz herangekommen.

4.5 Schritt 4: Von der 2D-Vorlage zur 3D-Szene (David)

Zur Generierung der 3D-Szene haben wir uns für HunyuanWorld-Mirror [16] entschieden. Die Funktionsweise des Modells ist, die Spiegelung der Input Bilder als 3D Szene. Wir haben das Modell recht schnell auf Palma zum Laufen bekommen und von dort aus erstmal intensiv auf die verschiedensten Input Bildern getestet.

Wir testeten das Modell auf Bildern aus der realen Welt und Bildern aus digitalen Welten. Videospiele eigneten sich, da es dort schon frei bewegbare virtuelle Welten gab, die wir versucht haben, wieder zu erzeugen. Unsere ersten Versuche mit dem Modell sind dabei fehlgeschlagen, das heißt wir haben es nicht geschafft die zu modellierte Welt aus den Bildern wieder zu Spiegeln.

In Abbildung 38 wurde ein einzelnes Bild als Input an HYW-Mirror gegeben, dabei ist das Bild erfolgreich aus der Perspektive aus, der es genommen wurde, widergespiegelt in einer 3D Szene. Jedoch zerfällt diese 3D Szene komplett, sobald wir es aus einer anderen Perspektive betrachten (Abb. 39).



Abbildung 38

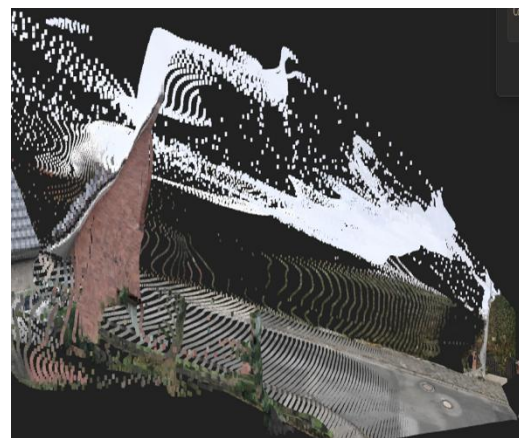


Abbildung 39

In den Abbildungen 40 und 41 sehen wir deutlich, dass das Modell ausreichend gute Übergänge zwischen den Input Bildern benötigt, um befriedigende 3D Szenen zu generieren.

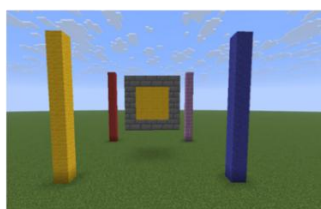
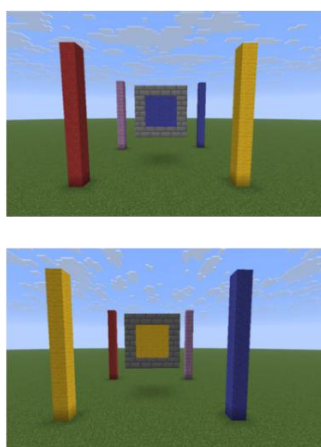
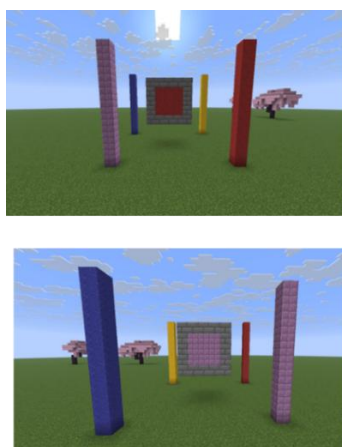


Abbildung 40 – Eingabebilder

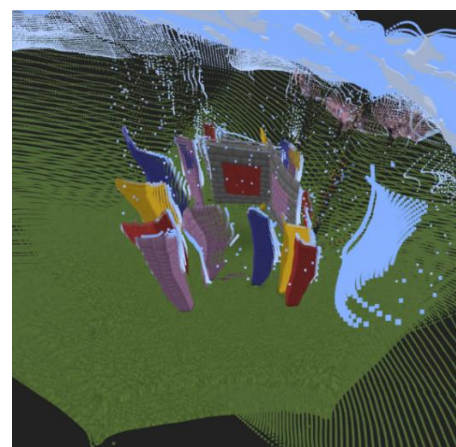


Abbildung 41 – generierte Punktwolke

Im Allgemeinen kann man somit sagen, dass das Modell sich nur auf die Input Bilder fokussiert und nicht eigene Strukturen versucht zu generieren, die nicht auf den Bildern erkennbar sind.

Unsere Beobachtungen nach dem ersten Testen mit HYW sind, dass das Modell als Eingabe folgendes braucht:

- mehrere verschiedene Perspektiven
- gute Übergänge zwischen den verschiedenen Perspektiven

sobald wir dies ermöglicht bekommen, dann bekommen wir ein, unserer Meinung nach, gutes Ergebnis mit HYW-Mirror.

Dies war auch der Grund, weswegen wir in der endgültigen Pipeline LTX-2 [6] nutzen, da wir mit den generierten Videos eine passende Eingabe erzeugen konnten. Dabei werden Bilder von den Input Video gesampelt und HunyuanWorld-Mirror versucht dann abhängig von den Bildern eine geeignete 3D Repräsentation zu generieren. So war HunyuanWorld-Mirror ein geeignetes Tool, welches in der Lage war die meisten Räume, die wir generieren wollten, zu generieren.

4.6 Schritt 5: Post-Processing und Umsetzung in Unity (Jonas)

Der Output von HunyuanWorld-Mirror (4.5) [16] war eine PLY-Punktwolke mit typischerweise ein bis zehn Millionen Punkten. Die Punktwolke bot noch keine zusammenhängende Oberfläche, die als begehbare 3D-Szene verwendet werden konnte. Ziel des Post-Processings war es, aus den rohen Punktwolken ein texturiertes Mesh zu erzeugen, welches in Unity [19] betreten werden konnte.

Dazu haben wir ein Python-Skript entwickelt, das den Großteil der Nachbereitung übernimmt und eine FBX-Datei mit einer dazugehörigen Textur ausgibt, welche einfach in Unity importiert werden konnte. Mit wenigen manuellen Ergänzungen in Unity konnte man anschließend die generierte Szene, zumindest mit Maus und Tastatur, erkunden.

4.6.1 Verarbeitung der Punktwolke mit Open 3D (Jonas)

Die Verarbeitungspipeline hat sich in die folgenden Schritte gegliedert:

Outlier Removal

Die generierten Punktwolken enthielten vereinzelte Ausreißer, die weit von der eigentlichen Szene entfernt lagen. Diese haben wir entfernt, indem wir für jeden Punkt die mittlere Distanz zu seinen Nachbarn berechnet haben. Aus allen Distanzwerten wurden ein globaler Mittelwert und eine Standardabweichung bestimmt. Alle Punkte, deren mittlere Nachbardistanz den Schwellenwert von Mittelwert plus konfigurierten Vielfachen der Standardabweichung überschritten hat, wurden als Ausreißer entfernt. (vgl. [27])

Die konkreten Filterparameter haben wir im Skript adaptiv an die Größe und Dichte der jeweiligen Punktwolke angepasst. Für die typischen Punktwolken mit ein bis zehn Millionen Punkten lagen die Werte bei 20 bis 30 betrachteten Nachbarn und einem erlaubten Vielfachen von 1,5 bis 2. Zudem haben wir die Punktwolke noch verdichtet, um glattere Oberflächen zu erzeugen.

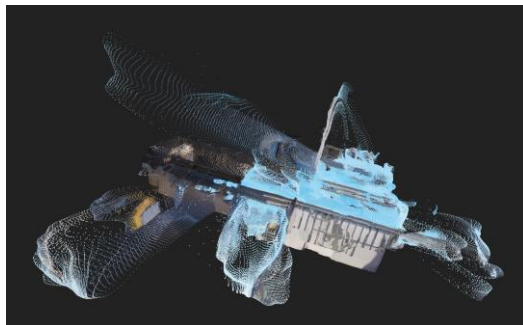


Abbildung 42 – vor dem Entfernen

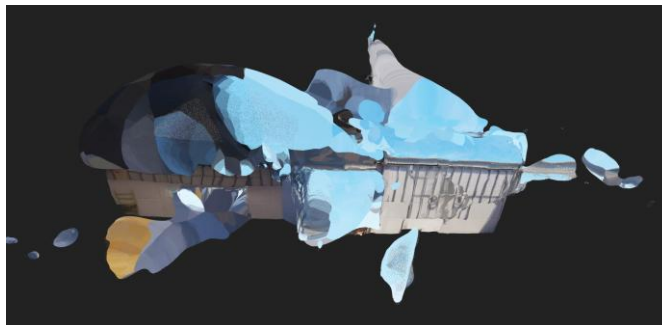


Abbildung 43 – nach dem Entfernen und Verdichten

Poisson Surface Reconstruction

Für die eigentliche Mesh-Rekonstruktion haben wir uns für die Poisson-Methode entschieden, da diese im Vergleich zum ebenfalls evaluierten Ball-Pivoting-Algorithmus (3.3.2) deutlich bessere wasserdichte Netze erzeugen konnte.

Poisson verwendet eine globale Indikatorfunktion und leitet daraus eine geschlossene Oberfläche ab. Open3D [17] bot eine native Funktion zur Verwendung der Methode an, was die Implementierung stark vereinfacht hat [27].

Die Poisson-Oberflächenrekonstruktion benötigt zwingend Normalenvektoren, daher haben wir, sollten keine Normalen vorhanden sein, diese mit der nativen Open3D-Funktion `estimate_normals` geschätzt und der ebenfalls nativen Open3D-Funktion `orient_normals_consistent_tangent_plane` konsistent ausgerichtet. [25]

Die Poisson Rekonstruktion hat danach für jeden Punkt bestimmt, ob dieser innerhalb oder außerhalb der Oberfläche lag. An der so extrahierten Isofläche wurde das eigentliche Mesh generiert. Dadurch hat die Methode eine geschlossene Oberfläche erzeugt, die auch bei unregelmäßigen Punktwolken sehr gut funktioniert (Abb. 44).

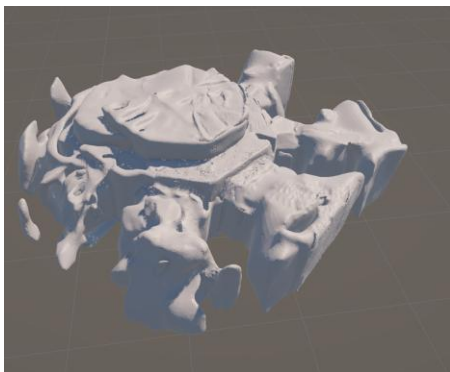


Abbildung 44

Mesh-Vereinfachung

Das von Poisson erzeugte Mesh enthielt eine sehr hohe Anzahl an Dreiecken, die für die Echtzeitdarstellung in Unity nicht notwendig war. Zudem waren die Oberflächen aufgrund der etwas unpräzisen Punktwolken sehr rau. Wir haben daher das Mesh auf etwa zehn Prozent der ursprünglichen Dreieckszahl reduziert. Dadurch wurden die Ungenauigkeiten geglättet, ohne sichtbare qualitative Einbuße zu verursachen (Abb. 45 vs. Abb. 46). Den Vereinfachungsfaktor haben wir experimentell bestimmt; er konnte je nach Punktwolke variieren.



Abbildung 45 – ohne Vereinfachung



Abbildung 46 – mit Vereinfachung

UV-Generierung und Textur-Baking

Wie bereits in Abschnitt 3.3.3 angesprochen, enthielt das rekonstruierte Mesh zunächst keine Farbinformationen (Abb. 44). Um die Farben der originalen Punktwolke auf das Mesh zu übertragen, haben wir ein Textur-Baking-Verfahren implementiert. Im Gegensatz zu der zuvor evaluierten Vertex-Farben-Methode (3.3.3) lieferte diese Methode eine deutlich höhere Qualität.

Um die Textur generieren zu können, haben wir zunächst mit der Bibliothek xatlas [28] für das Mesh UV-Koordinaten generiert. So wurde die Oberfläche des dreidimensionalen Netzes in einen zweidimensionalen Raum projiziert und jedem Vertex eine Position in der späteren Textur zugeordnet.

Zum Erzeugen der eigentlichen Textur (Abb. 47), haben wir für jedes Pixel der Textur die entsprechende dreidimensionale Position berechnet. Mithilfe eines KD-Baums wurde dann der nächstgelegene Punkt in der originalen Punktwolke gesucht und dessen Farbwert in das entsprechende Texturpixel geschrieben. Lücken in der Textur, die durch nicht abgedeckte Bereiche entstanden, wurden abschließend iterativ durch Mittelung der Farbwerte benachbarter Pixel aufgefüllt.

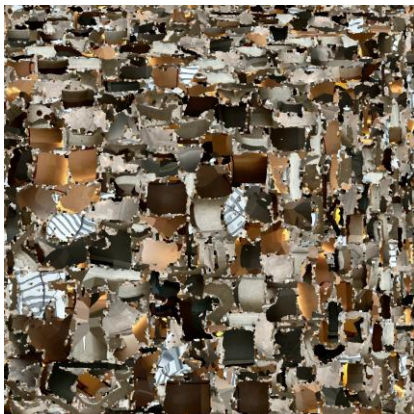


Abbildung 47



Abbildung 48 – Textur angewandt

FBX-Export

Das fertige Mesh wurde zusammen mit der generierten Textur als OBJ-Datei exportiert. Zusätzlich wurde eine MTL-Materialdatei erzeugt, welche die Zuordnung zwischen Mesh und Textur definierte. Da Unity das OBJ-Format nur eingeschränkt unterstützte und insbesondere

die Anwendung eines Mesh-Colliders nur mit FBX-Dateien möglich war, wurde die OBJ-Datei im letzten Schritt der Pipeline über ein Blender-Skript in eine FBX-Datei konvertiert.

4.6.2 Import und Interaktion in Unity (Jonas)

Nach dem Export aus der Post-Processing-Pipeline mussten die erzeugten Dateien, FBX-Mesh, MTL-Materialdatei und PNG-Textur, in Unity [19] importiert werden. In Unity mussten wir zudem noch ein paar manuelle Schritte ausführen, um die Szene betreten zu können. [29]

Import und Platzierung

Die drei Ausgabedateien wurden in den Assets-Ordner des Unity-Projekts kopiert, woraufhin Unity diese automatisch erkannte und importierte. Das FBX-Objekt wurde anschließend in die Szene gezogen und dort positioniert. Dabei fiel auf, dass die importierten Modelle in der Regel auf dem Kopf standen und um etwa 177 Grad rotiert werden mussten. Dieses Verhalten trat bei den meisten Modellen konsistent auf und war unabhängig vom Post-Processing; die genaue Ursache konnte nicht ermittelt werden.

Shader und Darstellung

Bei unseren ersten Tests mit Texturen zeigte sich, dass die Textur des Netzes, aufgrund des Backface Cullings, nur von einer Seite sichtbar war (Abb. 49, Abb. 50). Dies ist von Unity beabsichtigt um Rechenleistung zu sparen, da die generierten Meshes jedoch keine einheitliche Ausrichtung der Flächennormalen garantierten, führte dies zu unsichtbaren Flächen innerhalb der Szene.



Abbildung 49 - ohne Shader außen

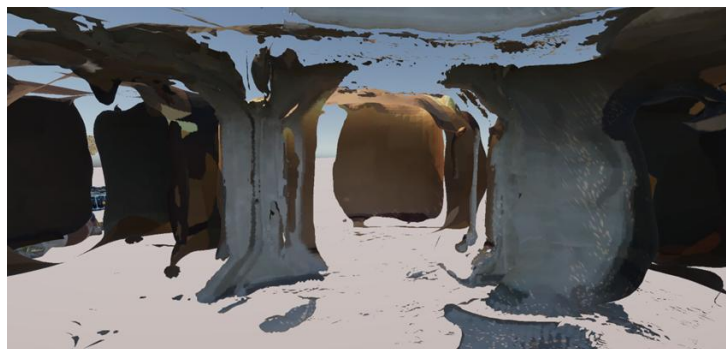


Abbildung 50 - ohne Shader innen

Um das Problem zu lösen, haben wir einen eigenen simplen Shader geschrieben. Der entscheidende Unterschied zum Standard-Shader lag in der Anweisung „Cull Off“, die das

Backface Culling deaktiviert und somit für das Rendern der Dreiecksrückseiten gesorgt hat. Da die Texturen der generierten Netze bereits den beleuchteten Zustand abdecken, haben wir den Shader als Unlit implementiert.

Um den Shader auf das Netz anwenden zu können, haben wir ein neues Material erstellt, diesem der eigene Shader zugewiesen und die generierte Textur als Base Map hinterlegt. Anschließend wurde das Material auf das importierte Mesh-Objekt angewendet.

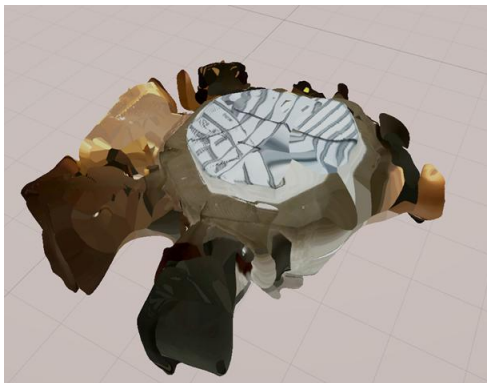


Abbildung 51 – mit Shader außen



Abbildung 52 - mit Shader innen

Kollisionsabfrage

Damit sich der Spieler nicht einfach durch Wände und Boden durchbewegen konnte, haben wir dem Mesh-Objekt ein Mesh Collider als Komponente hinzugefügt (Abb. 53). Dabei war es wichtig, die Option Convex deaktiviert zu lassen, da ein konvexer Collider den gesamten Raum als soliden Block behandelt hätte und man sich nicht innerhalb des Raumes hätte bewegen können. Wie bereits in Abschnitt 3.3.1 beschrieben, funktionierte der Mesh Collider nativ nur mit FBX-Dateien, was ein wesentlicher Grund für die Formatkonvertierung über Blender in der Verarbeitungspipeline war.

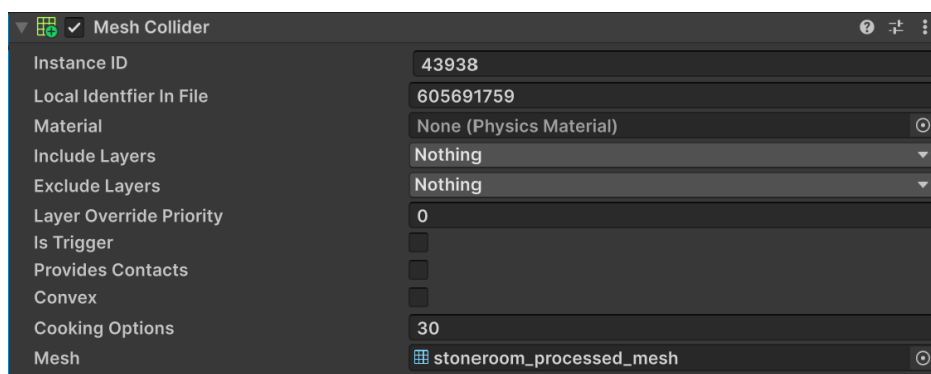


Abbildung 53

First-Person-Controller

Um die Szene wirklich betreten zu können, haben wir uns dazu entschieden einen First-Person-Controller [29] hinzuzufügen. So konnte die VR-Erfahrung zumindest am Bildschirm imitiert werden (Abb. 54). Dazu haben wir ein kostenlos verfügbaren First-Person-Controller aus dem Unity Asset Store eingesetzt, der sehr einfach hinzugefügt werden konnte.

Zusätzlich haben wir ein eigenes Skript ergänzt, das es ermöglichte, die Kollision zwischen dem Spieler und einem auswählbaren Mesh per Knopfdruck ein- und auszuschalten. Dies war notwendig, um den Spieler initial in den geschlossenen Raum hineinbewegen zu können.

So war es möglich die verschiedenen generierten Szenen in ein Unity-Projekt zu importieren und von Raum zu Raum zu gehen. Es mussten lediglich die Kollisionen kurzzeitig deaktiviert werden, damit man in den nächsten Raum gelangen konnte.

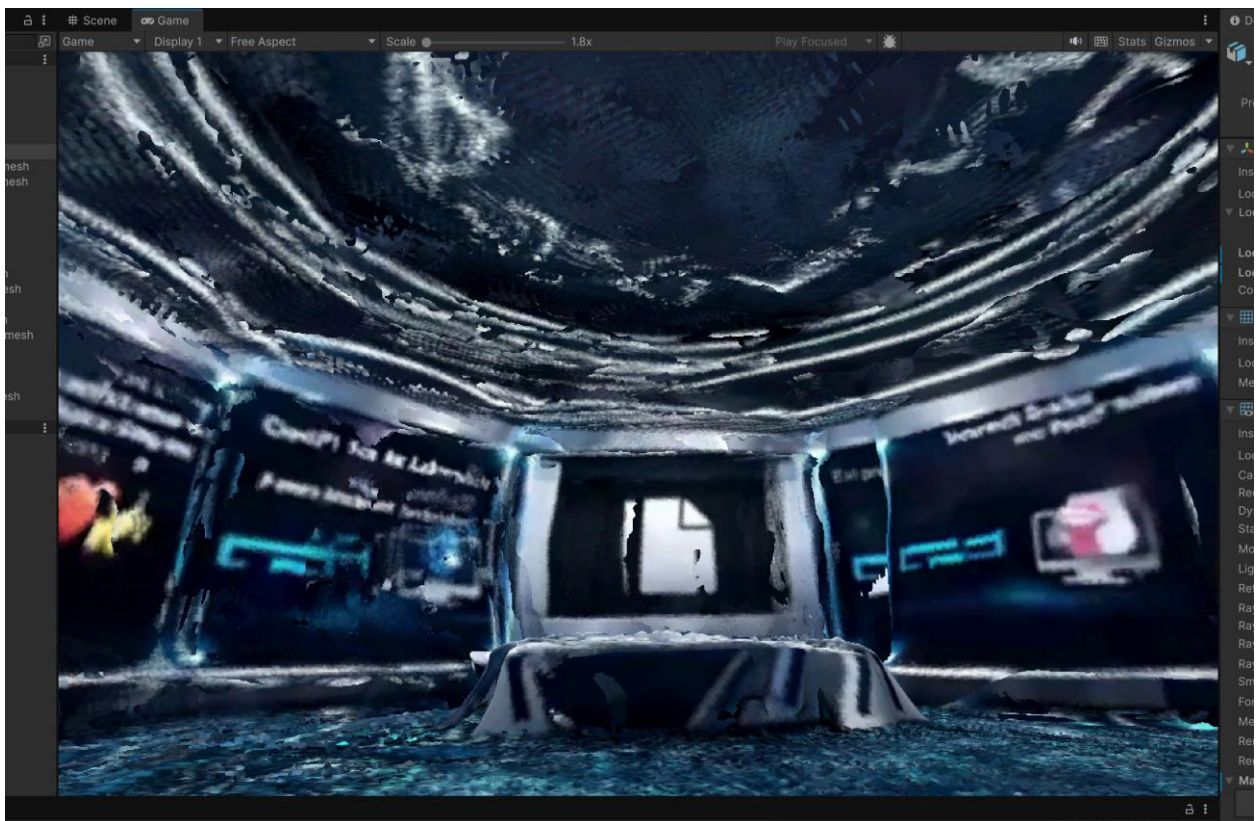


Abbildung 54

5. Ergebnisse der finalen Pipeline

In diesem Kapitel werden ein Beispiel Endergebnis, das von der in Kapitel 4 vorgestellten finalen Pipeline erzeugt wurde, kurz vor.

Schritt 1 – Promptverbesserung

Die ursprüngliche Raumbeschreibung wurde mit GLM4.7 [1] (4.2.1) in zwei optimierte Prompts überführt: einen für die Bildgenerierung mit Z-Image und einen für die anschließende Videoerzeugung mit LTX-2 [6]. Die Laufzeit betrug etwa 3 min. auf der A100.

erstellter Bild-Prompt (4.2.2):

„A photorealistic, wide-angle architectural shot of a hexagonal-shaped modern interior room. The entire ceiling is a structural glass skylight, flooding the space with bright natural light. The floor is polished grey granite. The raw concrete walls feature three other small and empty rooms, wide pathways leading to other rooms, each framed by large rounded square shaped architectural openings. The atmosphere is clean, minimalist, and airy, with sharp geometric lines. High definition, 8k, cinematic warm lighting. Other than the glass ceiling, there is no view outside“

erstellter Video-Prompt (4.2.3):

"The scene begins with the camera fixed at the exact geometric center of the given room shaped as a perfect heptagon, immediately initiating a perfectly smooth, constant-speed 360-degree rotation around its vertical axis with no acceleration, no deceleration, and no lateral movement. The camera remains fully stabilized, level, and locked at eye height, rotating like a precision-controlled gimbal to reveal each wall sequentially in equal intervals. The rotation completes one uninterrupted full circle with no camera shake, no motion blur distortion, and no environmental deformation. “

Schritt 2 – Bildgenerierung mit Z-Image

Aus dem optimierten Bildprompt erzeugte Z-Image [2] (4.3) ein Einzelbild (Abb. 55) des beschriebenen Raumes. Die Laufzeit betrug etwa 3 min auf der H200.



Abbildung 55

Schritt 3 – Videogenerierung mit LTX-2

Das generierte Bild diente zusammen mit dem in Schritt 1 erstellten Videoprompt als Eingabe für LTX-2 [6] (4.4). Der Videoprompt beschrieb eine gleichmäßige 360-Grad-Kamerarotation um die vertikale Achse im Zentrum des Raumes. LTX-2 erzeugte daraus ein Video, das den Raum aus verschiedenen Blickwinkeln zeigte und so die notwendigen Perspektiven für die 3D-Rekonstruktion lieferte. Die Laufzeit betrug etwa 2 min im LTX-Playground [7].



Abbildung 56

(Das Video-Ergebnis liegt in Sciebo unter *Projektseminar_Textto3D/Bild-Video-Generierung/Videos/LTX-2/Image-To-Video/Finale-Pipeline-Video*)

Schritt 4 – 3D-Rekonstruktion mit HunyuanWorld-Mirror

Aus dem Video rekonstruierte HunyuanWorld-Mirror [16] (4.5) eine Punktwolke des Raumes mit knapp einer Millionen Punkte. Die Laufzeit betrug etwa 8 min auf der H200.



Abbildung 57 – generierte Punktwolke

Schritt 5 – Post-Processing und Unity.

Die Punktwolke wurde, mit der in Abschnitt 4.6.1 beschriebenen Pipeline zu einem Mesh mit Textur (Abb. 58) verarbeitet, in das FBX-Format konvertiert. Danach konnte der generierte Raum in Unity [19] importiert und nach Einrichtung des Shaders, des Mesh Colliders und des First-Person-Controllers [29] betreten und erkundet werden (Abb. 59, Abb. 60). Die Laufzeit betrug ca. 18h mit 48 Cores auf requeue (CPU).



Abbildung 58 – finale Textur

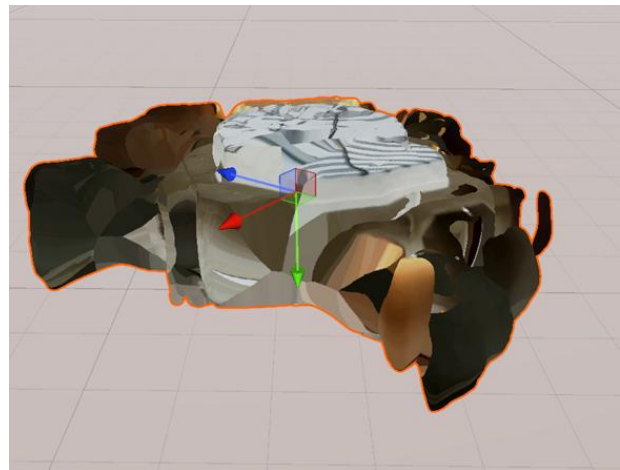


Abbildung 59 – Ergebnis in Unity



Abbildung 60 – In First Person im betretbaren Raum

6. Fazit (Franka)

Im Laufe des Projektseminares haben wir uns tiefer mit der in Betrieb Setzung, Nutzung, sowie Verknüpfung von KI-Modellen auseinandergesetzt und eine Optimierung jedes Schrittes der Pipeline angestrebt.

Dennoch gab und gibt es immer noch Limitationen bezüglich der vorgeschlagenen Pipeline. Unter anderem müssen alle vorgeschlagenen Modelle noch einzeln manuell ausgeführt werden und sind nicht in einer Anwendung verbunden. Außerdem wurde die gesamte Pipeline hauptsächlich für Welten, welche von den Germanistikstudenten gewünscht wurden, getestet. Dies umfasste ausschließlich geschlossenen Räume, die häufig im futuristischen Stil gehalten wurden. Es ist davon auszugehen, dass bei der Generierung von weitläufigeren Welten mehr Probleme auftreten würden. Auch das Testen der Ergebnisse mittels VR war aufgrund der zeitlichen Limitierung nicht mehr möglich.

Abschließend lässt sich dennoch sagen, dass wir durch die Verbindung der vorgestellten Modelle und Methoden ein für uns zufriedenstellendes Ergebnis für den gegebenen Zeitrahmen erzielt haben.

Literaturverzeichnis

- [1] zai-org, „GLM-4.7: huggingface“, 29 Januar 2025. [Online]. Available: <https://huggingface.co/zai-org/GLM-4.7>. [Zugriff am 3 März 2026].
- [2] Z-Image Team, Alibaba Group, „Z-Image: arxiv“, 8 Dezember 2025. [Online]. Available: <https://arxiv.org/pdf/2511.22699>. [Zugriff am 2 März 2026].
- [3] Black Forerst Labs, „flux-2: bfl“, 3 März 2026. [Online]. Available: <https://bfl.ai/blog/flux-2>. [Zugriff am 3 März 2026].
- [4] Z. Huang, Y.-C. Guo, H. Wang, R. Yi, L. Ma, Y.-P. Cao und L. Sheng, „MV-ADAPTER: arxiv“, 4 Dezember 2024. [Online]. Available: <https://arxiv.org/pdf/2412.03632>. [Zugriff am 4 März 2026].
- [5] S. Tang, F. Zhang, J. Chen, P. Wang und Y. Furukawa, „MVDiffusion: arxiv“, 25 Dezember 2023. [Online]. Available: <https://arxiv.org/pdf/2307.01097>. [Zugriff am 3 März 2026].
- [6] „LTX-2: arxiv“, 6 Januar 2026. [Online]. Available: <https://arxiv.org/pdf/2601.03233>. [Zugriff am 3 März 2026].
- [7] Lightricks, „ltx.studio“, 2 März 2026. [Online]. Available: <https://app.ltx.studio/ltx-2-playground/i2v>. [Zugriff am 2 März 2026].
- [8] Z. Xie, „WorldGen: github“, 11 Januar 2026. [Online]. Available: <https://github.com/ZiYang-xie/WorldGen>. [Zugriff am 3 März 2026].
- [9] Black Forest Labs, „Flux1ai“, März 3 2026. [Online]. Available: <https://flux1ai.com/de>. [Zugriff am 3 März 2026].
- [10] L. Piccinelli, C. Sakaridis, M. Segu, Y.-H. Yang, S. Li, W. Abbeloos und L. Van Gool, „UniK3D: github“, 15 September 2025. [Online]. Available: <https://github.com/lpiccinelli-eth/UniK3D>. [Zugriff am 3 März 2026].
- [11] H.-X. Yu, H. Duan, C. Herrmann, W. T. Freeman und J. Wu, „WonderWorld: arxiv“, 25 März 2025. [Online]. Available: <https://arxiv.org/pdf/2406.09394>. [Zugriff am 2 März 2026].
- [12] X. Tang, R. Li und X. Fan, „ZeroScene: arxiv“, 17 Februar 2026. [Online]. Available: <https://arxiv.org/pdf/2509.23607>. [Zugriff am 3 März 2026].
- [13] Z. Huang, Y.-C. Guo, X. An, Y. Yang, Y. Li, Z.-X. Zou, D. Liang, X. Liu, Y.-P. Cao und L. Sheng, „MIDI: arxiv“, 17 Juli 2025. [Online]. Available: <https://arxiv.org/pdf/2412.03558>. [Zugriff am 4 März 2026].

- [14] Z. Huang, Y.-C. Guo, X. An, Y. Yang, Y. Li, Z.-X. Zou, D. Liang, X. Liu, Y.-P. Cao und L. Sheng, „MIDI-3D: github,“ 12 Juni 2025. [Online]. Available: <https://github.com/VAST-AI-Research/MIDI-3D>. [Zugriff am 4 März 2026].
- [15] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. C. Berg und W.-Y. Lo, „Grounded-Segment-Anything: github,“ 5 September 2024. [Online]. Available: <https://github.com/IDEA-Research/Grounded-Segment-Anything>. [Zugriff am 4 März 2026].
- [16] C. Guo und T. Wang, „worldMirror1_0: tencent,“ 17 Oktober 2025. [Online]. Available: https://3d-models.hunyuan.tencent.com/world/worldMirror1_0/HYWorld_Mirror_Tech_Report.pdf. [Zugriff am 3 März 2026].
- [17] Open3D, „Open3D,“ 4 März 2026. [Online]. Available: <https://www.open3d.org>. [Zugriff am 4 März 2026].
- [18] Blender Foundation, „manual: blender,“ 26 Februar 2020. [Online]. Available: https://docs.blender.org/manual/de/2.82/getting_started/about/history.html. [Zugriff am 3 März 2026].
- [19] Unity Technologies, „unity,“ 4 März 2026. [Online]. Available: <https://unity.com/>. [Zugriff am 4 März 2026].
- [20] Epic Games, „Unreal Engine,“ 2 März 2026. [Online]. Available: <https://www.unrealengine.com/de>. [Zugriff am 2 März 2026].
- [21] Z. Huang, „MV-Adapter-T2MV-Anime: huggingface,“ 4 März 2026. [Online]. Available: <https://huggingface.co/spaces/huanngzh/MV-Adapter-T2MV-Anime>. [Zugriff am 4 März 2026].
- [22] Google, „gemini/2-5-flash: Google Cloud Docs,“ 4 März 2026. [Online]. Available: <https://docs.cloud.google.com/vertex-ai/generative-ai/docs/models/gemini/2-5-flash?hl=de>. [Zugriff am 4 März 2026].
- [23] B. Eren, „UnityPlyImporter: github,“ 25 Juli 2022. [Online]. Available: <https://github.com/bertaye/UnityPlyImporter>. [Zugriff am 2 März 2026].
- [24] Open3D, „tutorial: Open3D,“ 26 Februar 2026. [Online]. Available: https://www.open3d.org/docs/latest/tutorial/Advanced/surface_reconstruction.html. [Zugriff am 4 März 2026].
- [25] Open3D, „docs: Open3D,“ 1 August 2025. [Online]. Available: <https://www.open3d.org/docs/release/>. [Zugriff am 4 März 2026].

- [26] Lightricks, „Prompting Guide: ltx,“ 3 März 2026. [Online]. Available: <https://docs.ltx.video/api-documentation/prompting-guide>. [Zugriff am 3 März 2026].
- [27] Open3D, „pointcloud_outlier_removal. Open3D,“ 26 Februar 2026. [Online]. Available: https://www.open3d.org/docs/latest/tutorial/geometry/pointcloud_outlier_removal.html. [Zugriff am 4 März 2026].
- [28] J. Young und P. Rideout, „xatlas: github,“ 26 Juli 2022. [Online]. Available: <https://github.com/jpcy/xatlas>. [Zugriff am 4 März 2026].
- [29] Unity Technologies, „manual: unity,“ 3 März 2026. [Online]. Available: <https://docs.unity3d.com/6000.3/Documentation/Manual/>. [Zugriff am 4 März 2026].
- [30] S. S. Pasi, „Mini First Person Controller: unity assetstore,“ 6 Juli 2021. [Online]. Available: <https://assetstore.unity.com/packages/tools/input-management/mini-first-person-controller-174710>. [Zugriff am 4 März 2026].